MLflow 3.13 GenAI Workshop (Google Colab Pro Edition)

Run Prompt Registry, AI Governance & Local Open-Source LLMs (Llama 3.2) with Zero External API Keys and Zero Rate Limits!

In this hands-on lab, we run a full GenAI governance stack directly on your **Google Colab Pro GPU**:

1. **Local LLM Engine:** Run **Llama 3.2 (3B)** via **Ollama** on Colab's GPU (infinite calls, 0 rate limits).
2. **MLflow Prompt Registry:** Manage, version, and alias prompts as immutable assets.
3. **Typed Agents:** Build structured output agents with **Pydantic AI**.
4. **Evaluation & Optimization:** Benchmark prompts on golden datasets with `mlflow.genai.evaluate()`.

> **GPU Setting:** Go to **Runtime > Change runtime type > T4 GPU** (or L4 / V100). Standard T4 is fast, consumes minimal compute units (~1.5 units/hr), and has plenty of VRAM.

```
# Step 0: Check GPU availability in Colab
!nvidia-smi
```

```
Mon Nov  9 09:08:03 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |                    0 |
| N/A   43C    P8             13W /   70W |       0MiB /  15360MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

```
# Step 1: Install prerequisites with GPU detection, Ollama, and pull Llama 3.2 (3B)
!pkill -9 ollama || true
!apt-get update -qq && apt-get install -y -qq pciutils zstd
!curl -fsSL https://ollama.com/install.sh | sh

import subprocess, time, urllib.request

# Start Ollama daemon in the background
ollama_proc = subprocess.Popen(
    ["ollama", "serve"],
    stdout=subprocess.DEVNULL,
    stderr=subprocess.DEVNULL
)

# Wait until Ollama server is responding
print("Waiting for Ollama server to start...")
for _ in range(20):
    try:
        with urllib.request.urlopen("http://localhost:11434", timeout=1) as resp:
            if resp.status == 200:
                print("✓ Ollama server is ready!")
                break
    except Exception:
        time.sleep(1)

# Pull Llama 3.2 (3B) into GPU memory
!ollama pull llama3.2:3b

# Quick warmup call to force-load weights onto T4 VRAM
!ollama run llama3.2:3b "ready"
```

```
Preparing to unpack .../pci.ids_0.0~2024.03.31-1ubuntu0.1_all.deb ...
Unpacking pci.ids (0.0~2024.03.31-1ubuntu0.1) ...
Selecting previously unselected package libpci3:amd64.
Preparing to unpack .../libpci3_1%3a3.10.0-2build1_amd64.deb ...
Unpacking libpci3:amd64 (1:3.10.0-2build1) ...
```

```
Setting up pci.ids (0.0~2024.03.31-1ubuntu0.1) ...
Setting up libpci3:amd64 (1:3.10.0-2build1) ...
Setting up pciutils (1:3.10.0-2build1) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.9) ...
/sbin/ldconfig.real: /usr/local/lib/libtbb.so.12 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libhwloc.so.15 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind_2_5.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtcm_debug.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libumf.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtcm.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_loader.so.0 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind_2_0.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_level_zero.so.0 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbmalloc.so.2 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_level_zero_v2.so.0 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_opencl.so.0 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbmalloc_proxy.so.2 is not a symbolic link

>>> Cleaning up old version at /usr/local/lib/ollama
>>> Installing ollama to /usr/local
>>> Downloading ollama-linux-amd64.tar.zst
######################################################################## 100.0%
>>> Adding ollama user to video group...
>>> Adding current user to ollama group...
>>> Creating ollama systemd service...
WARNING: systemd is not running
>>> NVIDIA GPU installed.
>>> The Ollama API is now available at 127.0.0.1:11434.
>>> Install complete. Run "ollama" from the command line.
Waiting for Ollama server to start...
✓ Ollama server is ready!

I'm ready to help. What's on your mind?
```

```
# Step 2: Install MLflow 3.x and required libraries
!pip install -q "mlflow>=3.13" openai "pydantic-ai-slim>=1,<2" nest-asyncio loguru
```

```
# Step 3: Start MLflow Tracking Server with UI inside Colab
import subprocess, time

mlflow_proc = subprocess.Popen([
    "mlflow", "server",
    "--backend-store-uri", "sqlite:///mlflow.db",
    "--default-artifact-root", "./mlruns",
    "--host", "0.0.0.0",
    "--port", "5000"
])
time.sleep(3)

# Colab native port forwarding to view the UI in browser
try:
    from google.colab import output
    print("✅ MLflow Server running! Click the link below to view the MLflow UI:")
    output.serve_kernel_port_as_window(5000, anchor_text="👉 Click here to Open MLflow UI (Port 5000)")
except Exception as e:
    print("MLflow running at http://localhost:5000")
```

```
✅ MLflow Server running! Click the link below to view the MLflow UI:
Warning: This function may stop working due to changes in browser security.
Try `serve_kernel_port_as_iframe` instead.
👉 Click here to Open MLflow UI (Port 5000)
```

```
import difflib
import os
from typing import Any, Literal
import mlflow
import nest_asyncio
```

```
from openai import OpenAI
from pydantic import BaseModel
from pydantic_ai import Agent
from pydantic_ai.models.openai import OpenAIChatModel
from pydantic_ai.providers.openai import OpenAIProvider
from mlflow.genai.scorers import scorer

nest_asyncio.apply()

# Workshop Configuration
TRACKING_URI = "http://localhost:5000"
PROMPT_NAME = "workshop-ticket-triage"
EXPERIMENT_NAME = "mlflow-3-genai-workshop"
MODEL_NAME = "llama3.2:3b"

mlflow.set_tracking_uri(TRACKING_URI)
mlflow.set_experiment(EXPERIMENT_NAME)

# Connect OpenAI client to Colab's local Ollama endpoint (OpenAI-compatible)
local_llm_client = OpenAI(
    base_url="http://localhost:11434/v1",
    api_key="ollama",  # dummy key, 0 cost, 0 rate limits
)

print("✅ MLflow Tracking URI:", TRACKING_URI)
print("✅ Local Model:", MODEL_NAME, "via http://localhost:11434/v1")
```

```
WARNING:urllib3.connectionpool:Retrying (Retry(total=6, connect=6, read=7, redirect=7, status=7)) after connection broken by
WARNING:urllib3.connectionpool:Retrying (Retry(total=5, connect=5, read=7, redirect=7, status=7)) after connection broken by
WARNING:urllib3.connectionpool:Retrying (Retry(total=4, connect=4, read=7, redirect=7, status=7)) after connection broken by
WARNING:urllib3.connectionpool:Retrying (Retry(total=3, connect=3, read=7, redirect=7, status=7)) after connection broken by
WARNING:urllib3.connectionpool:Retrying (Retry(total=2, connect=2, read=7, redirect=7, status=7)) after connection broken by
✅ MLflow Tracking URI: http://localhost:5000
✅ Local Model: llama3.2:3b via http://localhost:11434/v1
```

```
# Verify local GPU model execution
response = local_llm_client.chat.completions.create(
    model=MODEL_NAME,
    messages=[{"role": "user", "content": "Explain what an MLflow AI Gateway is in one short sentence."}],
    temperature=0.2,
)
print("Local Llama 3.2 Output:")
print(response.choices[0].message.content.strip())
```

```
Local Llama 3.2 Output:
The MLflow AI Gateway is a cloud-based API gateway that enables secure, scalable, and efficient access to machine learning m
```

---## Part 1 — Prompt Registry: Versioning & PromotionInstead of hardcoding prompt strings into application code, we register them as immutable versioned assets with commit messages.

```
V1_TEMPLATE = """Classify this support ticket into one category:
PAYMENT, REFUND, RESCHEDULE, or OTHER.

Ticket: {{ ticket }}

Respond with the label only."""

prompt_v1 = mlflow.genai.register_prompt(
    name=PROMPT_NAME,
    template=V1_TEMPLATE,
    commit_message="v1: Initial minimal classification prompt",
    tags={"task": "ticket-triage", "backend": "colab-gpu"},
)
print(f"Registered {prompt_v1.name} version {prompt_v1.version}")
print(f"URI: {prompt_v1.uri}")
```

```
2026/11/09 09:10:16 INFO mlflow.store.model_registry.abstract_store: Waiting up to 300 seconds for prompt version to finish
Registered workshop-ticket-triage version 1
URI: prompts:/workshop-ticket-triage/1
```

```
loaded_v1 = mlflow.genai.load_prompt(f"prompts:/{PROMPT_NAME}/{prompt_v1.version}")

sample_ticket = "I was charged twice for my subscription this month."
filled = loaded_v1.format(ticket=sample_ticket)
print("Formatted prompt preview:")
```

```
print(filled)
```

```
Formatted prompt preview:
Classify this support ticket into one category:
PAYMENT, REFUND, RESCHEDULE, or OTHER.

Ticket: I was charged twice for my subscription this month.

Respond with the label only.
```

```
V2_TEMPLATE = """You are an expert support ticket triage assistant.

Task: Classify the ticket into exactly one of:
- PAYMENT: billing, duplicate charges, invoices
- REFUND: money back, cancellations, chargebacks
- RESCHEDULE: move/change appointment or delivery date
- OTHER: general inquiries

Rules:
- Output exactly one uppercase label
- No explanation, punctuation, or extra commentary

Ticket: {{ ticket }}"""

prompt_v2 = mlflow.genai.register_prompt(
    name=PROMPT_NAME,
    template=V2_TEMPLATE,
    commit_message="v2: Explicit category definitions and formatting rules",
)
print(f"Registered {prompt_v2.name} version {prompt_v2.version}")
```

```
2026/11/09 09:10:17 INFO mlflow.store.model_registry.abstract_store: Waiting up to 300 seconds for prompt version to finish
Registered workshop-ticket-triage version 2
```

```
v1 = mlflow.genai.load_prompt(f"prompts:/{PROMPT_NAME}/1")
v2 = mlflow.genai.load_prompt(f"prompts:/{PROMPT_NAME}/{prompt_v2.version}")

diff = difflib.unified_diff(
    v1.template.splitlines(),
    v2.template.splitlines(),
    fromfile="v1",
    tofile="v2",
    lineterm="",
)
print("Git-Style Prompt Diff:")
print("\n".join(diff))
```

```
Git-Style Prompt Diff:
--- v1
+++ v2
@@ -1,6 +1,13 @@
-Classify this support ticket into one category:
-PAYMENT, REFUND, RESCHEDULE, or OTHER.
+You are an expert support ticket triage assistant.
+
+Task: Classify the ticket into exactly one of:
+- PAYMENT: billing, duplicate charges, invoices
+- REFUND: money back, cancellations, chargebacks
+- RESCHEDULE: move/change appointment or delivery date
+- OTHER: general inquiries
+
+Rules:
+- Output exactly one uppercase label
+- No explanation, punctuation, or extra commentary

 Ticket: {{ ticket }}
-
-Respond with the label only.
```

```
# Promote v2 to production alias
mlflow.genai.set_prompt_alias(
    PROMPT_NAME, alias="production", version=prompt_v2.version
)

prod_prompt = mlflow.genai.load_prompt(f"prompts:/{PROMPT_NAME}@production")
print(f"✅ Alias '@production' now resolves to version {prod_prompt.version}")
```

```
✅ Alias '@production' now resolves to version 2
```

---## Part 2 — Combining Prompt Registry with Local LLM InferenceThe application queries the governed prompt by alias (`@production`) and passes it directly to our local GPU-backed Llama 3.2 model.

```
def classify_ticket(ticket: str, prompt_uri: str | None = None) -> str:
    uri = prompt_uri or f"prompts:/{PROMPT_NAME}@production"
    prompt = mlflow.genai.load_prompt(uri)

    response = local_llm_client.chat.completions.create(
        model=MODEL_NAME,
        messages=[{"role": "user", "content": prompt.format(ticket=ticket)}],
        temperature=0.1,
    )
    return response.choices[0].message.content.strip()


test_tickets = [
    "I was charged twice for one hotel night.",
    "Need to move my dental appointment to next Thursday.",
    "I want my money back for a cancelled flight.",
    "What are your customer support working hours?",
]

print(f"{'TICKET':<55} | {'CLASSIFICATION'}")
print("-" * 72)
for t in test_tickets:
    print(f"{t:<55} | {classify_ticket(t)}")
```

```
TICKET                                                  | CLASSIFICATION
------------------------------------------------------------------------
I was charged twice for one hotel night.                | PAYMENT
Need to move my dental appointment to next Thursday.    | RESCHEDULE
I want my money back for a cancelled flight.            | REFUND
What are your customer support working hours?           | OTHER
```

---## Part 3 — Typed Agents with Pydantic AI (Zero API Rate Limits)Pydantic AI creates structured agents where outputs are strictly validated into Python Pydantic models.

```
class TriageResult(BaseModel):
    category: Literal["PAYMENT", "REFUND", "RESCHEDULE", "OTHER"]
    reason: str


# Configure Pydantic AI with our local Colab model
pydantic_model = OpenAIChatModel(
    MODEL_NAME,
    provider=OpenAIProvider(
        base_url="http://localhost:11434/v1",
        api_key="ollama",
    ),
)


def triage_with_agent(
    ticket: str, prompt_uri: str = f"prompts:/{PROMPT_NAME}@production"
) -> TriageResult:
    prompt = mlflow.genai.load_prompt(prompt_uri)
    agent = Agent(
        pydantic_model,
        output_type=TriageResult,
        system_prompt="Classify customer support tickets accurately. Follow format rules.",
    )
    return agent.run_sync(prompt.format(ticket=ticket)).output


test_ticket = "I was charged twice for one hotel night."
result = triage_with_agent(test_ticket)
print("Structured Agent Output:")
print(f"Ticket   : {test_ticket}")
print(f"Category : {result.category}")
print(f"Reason   : {result.reason}")
```

```
Structured Agent Output:
Ticket   : I was charged twice for one hotel night.
Category : PAYMENT
Reason   : duplicate charges
```

---## Part 4 — Benchmark & Evaluation on Golden DatasetWe evaluate version 1 and version 2 side-by-side using `mlflow.genai.evaluate()` and log the runs to MLflow.

```
EVAL_DATA = [
    {
        "inputs": {"ticket": "Charged twice for one hotel night."},
        "expectations": {"expected_response": "PAYMENT"},
    },
    {
        "inputs": {"ticket": "Need to move my appointment to next week."},
        "expectations": {"expected_response": "RESCHEDULE"},
    },
    {
        "inputs": {"ticket": "I want my money back for a cancelled flight."},
        "expectations": {"expected_response": "REFUND"},
    },
    {
        "inputs": {"ticket": "What are your opening hours on weekends?"},
        "expectations": {"expected_response": "OTHER"},
    },
]

@scorer
def category_match(outputs: Any, expectations: dict[str, Any]) -> float:
    pred = str(outputs).strip().upper().strip(".")
    expected = expectations["expected_response"]
    return float(expected in pred or pred == expected)

@scorer
def label_guidelines(outputs: Any) -> float:
    val = str(outputs).strip().upper().strip(".")
    valid = {"PAYMENT", "REFUND", "RESCHEDULE", "OTHER"}
    return float(val in valid)

def make_predict_fn(prompt_uri: str):
    def predict_fn(ticket: str) -> str:
        return classify_ticket(ticket, prompt_uri=prompt_uri)
    return predict_fn

print(f"Evaluation Dataset: {len(EVAL_DATA)} golden examples ready.")
```

```
Evaluation Dataset: 4 golden examples ready.
```

```
# Side-by-side evaluation
results = {}
for version in [1, 2]:
    uri = f"prompts:/{PROMPT_NAME}/{version}"
    with mlflow.start_run(run_name=f"eval-v{version}-local") as run:
        eval_result = mlflow.genai.evaluate(
            data=EVAL_DATA,
            predict_fn=make_predict_fn(uri),
            scorers=[category_match, label_guidelines],
        )
        results[version] = eval_result
        print(f"v{version} Metrics: {eval_result.metrics}")
        print(f"  → Run ID: {run.info.run_id}")
```

```
2026/11/09 09:10:18 INFO mlflow.models.evaluation.utils.trace: Auto tracing is temporarily enabled during the model evaluati
2026/11/09 09:10:18 INFO mlflow.genai.utils.data_validation: Testing model prediction with the first sample in the dataset.
Evaluating: 100%  4/4 [Elapsed: 00:04, Remaining: 00:00] [predict_fn: 25%, scorers: 75%]
View evaluation results in MLflow
2026/11/09 09:10:24 INFO mlflow.genai.utils.data_validation: Testing model prediction with the first sample in the dataset.
v1 Metrics: {'label_guidelines/mean': np.float64(1.0), 'category_match/mean': np.float64(0.5)}
  → Run ID: 16f10e018cce443cb5b901e827e9b105
Evaluating: 100%  4/4 [Elapsed: 00:00, Remaining: 00:00] [predict_fn: 83%, scorers: 17%]
View evaluation results in MLflow
v2 Metrics: {'category_match/mean': np.float64(1.0), 'label_guidelines/mean': np.float64(1.0)}
  → Run ID: abd3a1abfb7c4e85a9185bcc3c5e96d0
```

```
# Step 5: Register optimized version 3 and promote to production
V3_TEMPLATE = """You are an expert AI support ticket triage assistant.
Classify the customer ticket into exactly one category:
- PAYMENT: Billing, duplicate charges, payment failures, invoices
- REFUND: Money back, cancellations, chargeback requests
```

```
- RESCHEDULE: Moving appointment dates, changing delivery times
- OTHER: General inquiries, office hours, feedback

Rules:
- Output exactly one uppercase label only
- No extra punctuation or explanation

Ticket: {{ ticket }}"""

prompt_v3 = mlflow.genai.register_prompt(
    name=PROMPT_NAME,
    template=V3_TEMPLATE,
    commit_message="v3: High-accuracy production prompt with explicit boundary constraints",
)

# Promote v3 to production
mlflow.genai.set_prompt_alias(
    PROMPT_NAME, alias="production", version=prompt_v3.version
)

print(f"🎉 Promoted {prompt_v3.name} v{prompt_v3.version} to @production!")

# Re-evaluate production alias
with mlflow.start_run(run_name="eval-production-v3") as run:
    final_eval = mlflow.genai.evaluate(
        data=EVAL_DATA,
        predict_fn=make_predict_fn(f"prompts:/{PROMPT_NAME}@production"),
        scorers=[category_match, label_guidelines],
    )
    print(f"Production metrics: {final_eval.metrics}")
```

```
2026/11/09 09:10:25 INFO mlflow.store.model_registry.abstract_store: Waiting up to 300 seconds for prompt version to finish
2026/11/09 09:10:26 INFO mlflow.genai.utils.data_validation: Testing model prediction with the first sample in the dataset.
🎉 Promoted workshop-ticket-triage v3 to @production!
```

Evaluating: 100% 4/4 [Elapsed: 00:00, Remaining: 00:00] [predict_fn: 77%, scorers: 23%]

View evaluation results in MLflow

```
Production metrics: {'category_match/mean': np.float64(1.0), 'label_guidelines/mean': np.float64(1.0)}
```

---## Workshop SummaryYou ran the full GenAI governance stack completely on **Google Colab Pro GPU**:1. **Local LLM Engine:** Llama 3.2 (3B) with zero external API keys and zero rate limits.2. **Prompt Registry:** Immutable version control (v1, v2, v3), git-style diffs, and zero-downtime `@production` aliasing.3. **Typed Agents:** Pydantic AI structured extraction.4. **Evaluation:** Objective benchmark scoring in MLflow.Open your MLflow UI (click the link from Step 3) to view all runs, traces, and prompt versions!

```
Start coding or generate with AI.
```